

Vulnerability Patching via Structure-Aware Retrieval-Augmented Generation

Master's Thesis

TUM School of Computation,
Information and Technology

Institut für Informatik
Lehrstuhl III (I3)
Prof. Dr. Jana Giceva
Makreshanska

Boltzmannstr. 3
Room: 02.11.043
85748 Garching

<https://www.db.in.tum.de/>

Supervisor: Prof. Dr. Jana Giceva Makreshanska
Advisor: Fabian Wenz
Email: jana.giceva@in.tum.de
Phone: +49 (0) 89 / 289-17286
Starting date: April 2026

Context

0.1 Research Background

To meet compliance requirements, organizations must adhere to International Standards and regulations such as ISO/IEC 27001 [1]. This standard establishes best practices for data protection, cyber-resilience, and operational excellence. To accomplish that and improve development robustness, enterprises must anticipate and adapt to vulnerabilities and potential attacks. For software companies, this goal demands evolving software development life cycles to rapidly identify, generate, and deploy risk mitigation for vulnerabilities. One possible approach is to integrate automated systems that proactively generate patches for buggy programs.

With the advent and widespread adoption of Large Language Models (LLMs), a new venue for automated remediation emerged [?]. However, the current LLM approaches remain insufficient for security patching due to two primary failure reasons:

1. **Semantic Misunderstanding:** 84–94% of AI patches rely on localized fixes that fail to address root causes, making them significantly more fragile and easier to bypass than human-authored patches [2].
2. **Vulnerability Identification Failures:** Models frequently misidentify vulnerability contexts, with some retrieval tasks showing failure rates as high as 91.6% [3]. This leads to "semantic hallucinations", where a model might apply mitigation steps for the wrong operating system [3] or package version [4], creating a false sense of security while leaving the actual system exposed.

0.2 Problem Statement

These failures are aggravated by providing lengthy raw source code to the LLM. In these cases, vulnerability-relevant logic is diluted by implementation details and boilerplate code, thereby degrading the quality of security analysis [?]. Empirical results from the same work suggest that this code noise induces what the authors term Analysis Focus Drift, wherein the model performs a code quality review rather than a vulnerability analysis. Thus, there is an opportunity to develop methodologies that enrich security-relevant information while filtering out implementation noise. Moreover, although various studies show that knowledge bases improve LLM responses in security applications [?, ?, ?, 5], it is unclear which approach optimizes patch quality effectively in multi-agent environments.

0.3 Research Objectives

To evaluate the proposed framework, we address two primary research questions:

RQ1: How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?

RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

If within reach, we aim to also address the following complementary research questions:

CRQ1: How does the structure-aware RAG framework with a small model compare with existing approaches (e.g., zero-shot small models and AutoPatch)?

CRQ2: What is the computational and financial cost of the approach (e.g., token usage, storage complexity and size, end-to-end pipeline costs)?

Related Work

Automated Program Repair (APR) aims to fix functional and security defects, which are critical in large-scale software systems. With advances in LLM coding capabilities, systems such as RepairAgent [6] have demonstrated that autonomous LLM agents can resolve hundreds of bugs through iterative tool interaction. However, this process comes at a high computational cost, averaging 270,000 tokens per bug, raising concerns about the efficiency of these approaches.

Automated Vulnerability Repair (AVR), a subtask of APR, focuses on addressing software security vulnerabilities while minimizing dependence on domain experts [?]. However, APR techniques do not transfer seamlessly to AVR [?], necessitating specialized approaches. Systems such as FineNib [?] exemplify this complexity by incorporating formal static analysis artifacts, specifically CodeQL queries, as executable verification oracles during code generation. In contrast to approaches that rely on existing security tools, AutoPatch [?] proposes a system grounded through RAG and multiple agent interactions to achieve meaningful results. RAG was originally proposed to combine the flexibility of pre-trained parametric memory with the precision of non-parametric external knowledge [?], enabling models to consult a document index and reason beyond their training data. Despite these advantages, RAG-based approaches identified in this review depend exclusively on textual vector embeddings for retrieval, failing to capture deep structural signatures of vulnerabilities, such as taint flows, data dependency chains, and control flow anomalies. This limitation creates a representational gap, causing these systems to fall back on the inherent capacity of the generative model, assuming that scale is the main lever for repair quality. Consequently, this dependency results in high repair costs and structural logic errors [?].

TODO CVE vs CWE definition source and sink definition

Joern [7] is a static analysis tool that uses their own graph definition to represent the code, namely Code Property Graphs (CPGs). It consists of nodes that contain the code and a label; and a set of edges that can represent either an edge from abstract syntax tree, control flow, data dependency or control dependency. Joern supports the graph generation and exportation; and disposes of a language for graph traversal with the aim of searching a developer defined vulnerability. As a consequence, it requires a technical knowledge of the vulnerability and the required language, besides a deep understanding of how the code flows (e.g. sinks and sources). Regardless, it is a powerful tool that can have lower false positives than other tools like (dog tool name here)

Define graphs Prior to the recent advancements in LLM-based code understanding [?], static analysis techniques such as Code Property Graphs (CPGs) [?] were already well established. CPG is a code representation that merges abstract syntax trees, control flow graphs, and data flow graphs into a joint data structure, facilitating vulnerability detection through graph traversal. Although CPGs have been applied to vulnerability detection, their potential as a retrieval representation for automated vulnerability repair remains unexplored.

The proposed approach aims to ground retrieval in graph embeddings, using structural similarity as the primary retrieval signal, rather than relying only on semantic keyword matching.

Methodology

This work proposes a structure-aware RAG framework for automated vulnerability patching. Rather than relying only on the generative capacity of frontier models, this approach shifts part of the patching complexity to retrieval precision, better grounding the repair process.

The vulnerable code will be represented using graph embeddings to retrieve structurally similar patches from a knowledge base at inference time. Beyond retrieval, we will apply the graph edit distance [?, ?] to identify structural differences between vulnerable and patched code within the retrieved pairs. These structural differences serves as a grounding repair orientation for the LLM to remediate a flaw.

We investigated how this design leverages structural meaning encoded in graph representations to identify the lines that introduce security breaches.

Architecture

The architecture employed in this work is displayed in Figure1, where the box containing Δ represents our contribution. All the other components are derived from existing frameworks.

The system consists of a knowledge base (KB) and an agentic framework. The KB is constructed from datasets annotated with CVE (Common Vulnerabilities and Exposures) and CWE (Common Weakness Enumeration). Each entry contains the vulnerable/patched code pair and its structural differences extracted via subgraph matching.

The system will use an agentic framework common in industrial applications and similar to the AutoPatch [?] design and employ two specialized LLM agents:

- Patcher Agent: synthesizes code repairs by using the retrieved context in its prompt;
- Vulnerability Verifier Agent: receives the generated patch and verifies whether the previously spotted vulnerability persists.

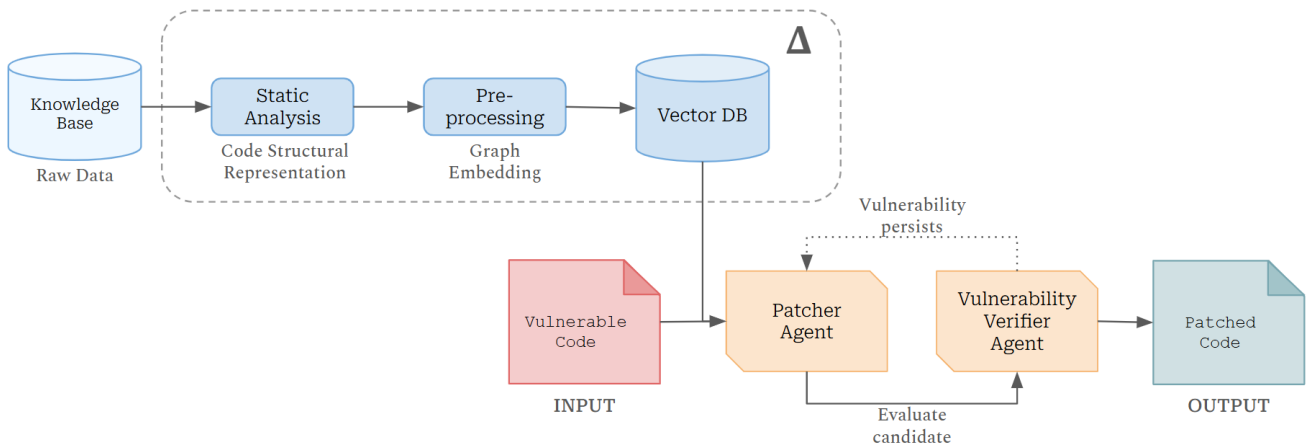


Figure 1: System architecture used in this work to verify the graph embedding usage for patching vulnerabilities, where the Δ -box is our contribution.

Data Pipeline

Graph Generation

This work utilizes Joern to generate the Code Property Graph (CPG) from a code snippet. For snippets that are not well-formed C/C++ programs, the pipeline first wraps the code with the minimum necessary

macros and type definitions (e.g. `typedef unsigned int u32, #define NULL ((void*)0)`) and writes a parseable source file. The C/C++ code files are exported to a binary graph with the Joern shell command `joern-parse --language newc`, where `newc` specifies the C++ specificities avoiding will-formed graphs. Then, the compiled graph is exported to GraphML format with the shell-command `joern-export --repr cpg --format g`. Each export generates a directory tree where each subdirectory contains an `export.xml` file with a sub-graph of the the whole CPG. The nodes can have different labels (e.g. `CALL`, `IDENTIFIER`, `CONTROL_STRUCTURE`), whereas the edges can belong to one of the following graph representation: Abstract Syntax Tree (AST), Control Flow Graph (CFG), Call Dependency Graph (CDG), and Program Dependency Graph (PDG).

The pipeline generates both *vulnerable* and *patched* code snippet graphs.

Graph Loading

The exported graph is imported into the pipeline using the NetworkX library as a `MultiDiGraph`. All the `export.xml` discovered by recursively exploring the subdirectories. Once the graph they store is imported to the pipeline, it is merged by reading each file with `nx.read_graphml` and updating the combined graph iteratively. The NetworkX library considers each graph, node and edges as a dictionary and update the data structure by adding one graph set of nodes and edges at a time. The Joern definition of the graph is, that is, the node and edge attributes, is conserved, therefore, the graph consistency depends on the Joern export. Once the graph is loaded, the following cleanups are applied:

- Nodes with the label `COMMENT` or `UNKNOWN` are removed
- Any remaining edges whose source or target were removed are pruned.

Graph Sampling

Once the CPG is loaded, it contains the full code, including the boilerplate added in the pipeline. Therefore it is necessary to isolate the subgraph related to the vulnerability from the irrelevant rest of the code. For that, a subsample is calculated from the difference between the vulnerable graph and the patched graph, the so called G_{vuln} . By doing so we expect to be able to generalize the search for the vulnerability removing the specificities of a given system that are inprint in the graph.

Bounded slicing To include context that may be relevant, a bounded search is performed with the goal of include any relevant settings in which the vulnerability occurs. The slice is a configurable neighbourhood expansion ($n \in \{0, 1, 2, 3\}$ hops). $n = 0$ is the original sampled graph, that is, only directly changed nodes.

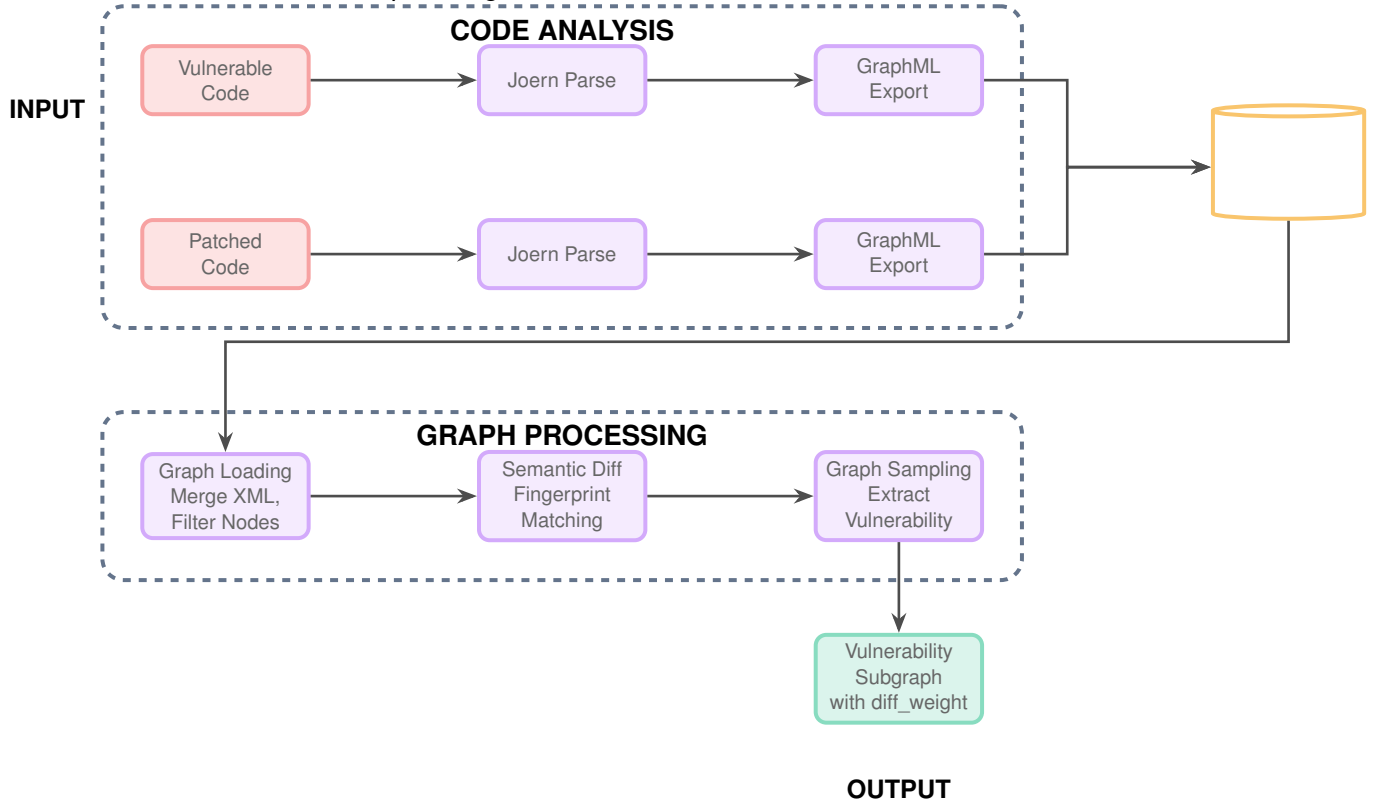
Semantic difference Before perform the sampling, each node needs to be deterministically identifiable to avoid different results depending on runs or randomness resulting from non-deterministic ID assigned by Joern across runs. The nodes in the sample are further enriched with semantic labels resulting of the two graphs comparison. Each node of the sample is labeled as either:

- Removed node: every node present in G_{orig} but absent in the G_{patch} .
- Fix-adjacent node: every node present in G_{orig} that is a removed node neighbor.
- Edge-changed node: every node attached to an edge which any of the nodes changed between the two graphs.

Besides the labelling, the G_{vuln} nodes also received weights.

Label	Weight
removed	1.0
fix_adjacent	0.8
edge_changed	0.6
context	0.2

These weights are directly proportional to the relation of the node to the vulnerability and will be integrated to the embedders for further pruning.



Embedders

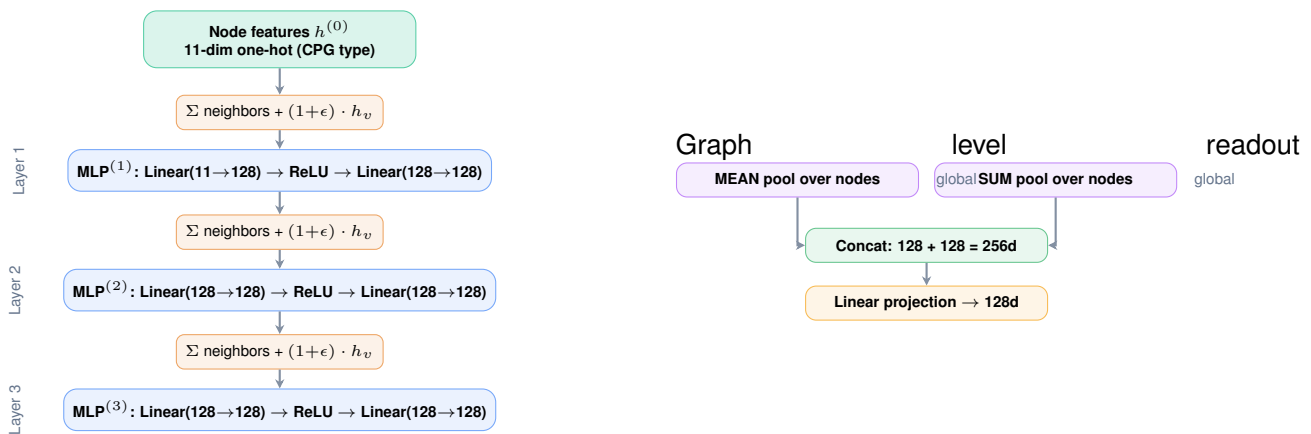
For reference, we started with deterministic embedders that work either with heuristics or with well-defined mathematical formulations. All embedders are **unsupervised**, that is, no labelled training data required. By default their output is L2-normalised vectors with a default dimension of 128, but configurable. Besides that, the embedding may have a PCA.

- **NetLSD** (*Network Laplacian Spectral Descriptor*) [8]. Computes the heat trace of the normalised graph Laplacian across a timescale that aims to get graph expressive representations similar to random walk models. It is node permutation-invariant and captures global spectral structure, adapting to scale encoding both local and global features, that is, it is scale-adaptable. It is also capable of comparing different size graphs. It supports weighted graphs. It was created to be an alternative to the NP-approach graph edit distance, being then efficiently computable, so the comparison calculation happens in a constant time.
- **WL** (*Weisfeiler-Lehman Kernel*). Iterative colour refinement over node types followed by weighted sum pooling and a linear projection. In this work, it was implemented in PyTorch Geometric with no external dependency. WL test iteratively : (1) aggregates the labels of nodes and their neighborhoods, and (2) hashes the aggregated labels into unique new labels
- **GIN** (*Graph Isomorphism Network*) [9] Message-passing neural Network that is as powerful as the Weisfeiler-Lehman graph isomorphism test, which evaluates whether two graphs are topologically identical. A Graph Neural Network that represents each node by recursively aggregating the neighboring representation. The node representation is updated at each layer k , by aggregating the representations of its neighbors via a sum aggregator, followed by an MLP:

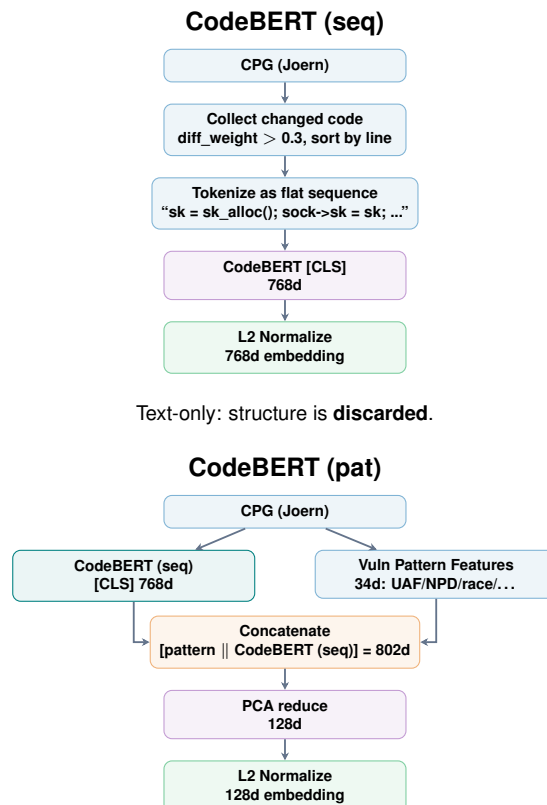
$$h_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right)$$

Where ϵ is either a learnable paramet or a fixed scalar (TODO: check what is the case here). $h_v^{(0)}$ is initialized from the node features. (TODO: explain network initialization). The number of layers is equivalent to the number of neighbor hops that will be encoded by each node. This choice is a trade-off of the amount of local structural pattern against the global one. Therefore it learns to embed the subtree in a low dimensional space, not only discriminating different structures but also mapping similar embeddings for similar structures and capturing dependencies between graph structures. Graph-level embeddings are obtained via dual pooling concatenating sum pooling and mean pooling outputs. The sum pooling preserves structural size information, while mean pooling makes the normalized representations normalized.

$$\mathbf{h}_G = \left[\sum_v h_v^{(K)} \parallel \frac{1}{|V|} \sum_v h_v^{(K)} \right]$$



- **Combined.** Concatenates NetLSD, WL and GIN descriptors ($3 \times 128 = 384$ dimensions) and reduces via PCA to the target dimension, by default 128. It aims to be most robust to individual embedder failures.



Fuses **graph structure** with semantics.

PCA and L2 normalization Discussion with experiments

FAISS and HNSW

Agent prompt iteration

The pipeline begins by generating a graph embedding of the input vulnerable code snippet. This embedding is used to query the KB for the most structurally similar fix. The AutoPatch dataset will be used as the primary source for the KB, providing a consistent baseline for our experiments.

Once the relevant entry is retrieved, the Patcher Agent enriches its prompt with the structural data and other relevant fields to generate a candidate patch. This output is then forwarded to a Vulnerability Verifier Agent for validation.

To manage the indices, the infrastructure requires a graph database or a specialized vector store.

As an ablation study, we would like to evaluate the performance of a small, open-source LLM as the patch generator, comparing its results with and without the KB context enrichment.

Evaluation

Evaluation will be conducted using a curated dataset of vulnerabilities drawn from the AutoPatch corpus.

Metrics. We report the following, all computed at $k=5$ unless stated otherwise:

- **hit@ k :** fraction of queries whose correct CVE appears in the top- k retrieved results (micro-averaged).
- **MRR:** mean reciprocal rank of the first correct CVE across all queries.
- **CWE Recall:** for each query, the fraction of same-CWE items retrieved in top- k , macro-averaged across CWE classes.
- **Effective dimensionality:** the number of PCA components needed to explain 90% of the embedding variance, measuring how many independent directions the embedder uses.
- **Retrieval Quality:** Measured via Hit@K, which calculates the fraction of true entities appearing in the top k results of the ranked list. This metric evaluates the performance of the Graph-based search.
- **Patch vulnerability verification:** Average loop counts will measure the number of iterations required before the vulnerability is no longer detected. Notably important for answering RQ2 and directly comparing results with AutoPatch.
- **Efficiency:** A comparative cost analysis (measuring tokens, as in RepairAgent, and USD, as in AutoPatch) against LLM-only solutions.
- **Patch Quality:** A qualitative assessment by industry security specialists on a representative sample of generated patches.
- **Storage complexity:** We will measure and compare the storage size of a KB using graph embeddings versus one using traditional text embeddings.

1 Dataset

- **AutoPatch dataset (evaluation benchmark).** We use AutoPatch as the main benchmark for end-to-end patching evaluation, enabling direct comparison with prior RAG-based vulnerability repair results.
- **CVEfixes dataset (retrieval knowledge base).**
 - **Origin.** CVEfixes is a public vulnerability-fix dataset built by linking CVE/CWE records to real source-code commits across open-source repositories (DOI: <https://doi.org/10.5281/zenodo.4476563>).
 - **Collection method.** The dataset mines vulnerability-fixing commits and stores aligned metadata in SQLite tables (e.g., `cve`, `cwe_classification`, `fixes`, `commits`, `file_change`, `method_change`). This provides both file-level snapshots (before/after) and method-level changes tied to the same vulnerability event.

- **Composition used in this thesis.** We retrieve C/C++ entries and use CVE/CWE labels as retrieval supervision metadata. We keep both graph artifacts (for structural retrieval) and source code (for LLM grounding and patch generation).

- **Observed CVEfixes file-level size in our local SQLite copy** (`data/cvefixes/CVEfixes.db`).

Metric on <code>file_change.code_before</code>	Value
Total rows in <code>file_change</code>	51,342
Non-null rows	51,342
Non-empty rows	51,342
Min length (characters)	1
Mean length (characters)	39,190.41
Max length (characters)	33,397,878
Rows with length $\geq 1,000$	37,553
Rows with length $\geq 10,000$	19,348
Rows with length $\geq 100,000$	3,017

- **Distribution buckets for `code_before` length (characters).**

Bucket	Rows
< 100	9,690
100–999	4,099
1,000–9,999	18,205
10,000–99,999	16,331
$\geq 100,000$	3,017

- **Method input vs file input in our pipeline.**

Aspect	Method-level input	File-level input (current)
Source unit	<code>method_change.code</code> (single function diff)	<code>file_change.code_before/code_after</code> (full file snapshots)
Typical context size	Compact; high signal, lower surrounding context	Larger context; includes surrounding logic and declarations
Graph export granularity	One CPG per changed method	One CPG per file pair (before/after) or merged method CPGs
Best use case	Fine-grained pattern retrieval and focused localization	End-to-end patch generation with complete file context
Trade-off	May miss file-scope interactions	Higher compute/storage and more potential boilerplate noise

Requirements for a Leakage-Safe Retrieval Split

The retrieval evaluation partitions the labelled pairs into an *index* set, which forms the searchable knowledge base, and a disjoint *query* set of held-out probes. Because the knowledge base simultaneously acts as the retrieval target, a careless partition can render the task either trivial — when a near-identical copy of the answer is already present in the index — or impossible — when no valid answer is present at all. To obtain an unbiased estimate of retrieval generalisation, the split must satisfy the following requirements.

- R1. **No index/query leakage.** No query and its ground-truth evidence may originate from an artefact that is also present, in identical or near-identical form, in the index; otherwise retrieval degenerates into recognising a memorised copy rather than generalising a vulnerability pattern. Leakage of this kind is a well-documented source of over-optimistic results both in general machine-learning pipelines [10] and, specifically, in machine-learning-for-security evaluations [11].
- R2. **Guaranteed positive support (retrievability).** Every query must have at least one valid positive in the index; otherwise the same-CVE $\text{Hit}@k$ and MRR are undefined for that query, since the correct item cannot be retrieved at any rank. The split therefore keeps at least one pair of each queried CVE inside the index.
- R3. **Non-trivial positives.** The retained positive must not be a duplicate or near-duplicate of the query. Duplicate and near-duplicate samples are pervasive in vulnerability datasets and are known to substantially inflate reported performance [12, 13].
- R4. **Representativeness (stratification).** The index and query sets preserve the CWE distribution of the corpus, so that per-class metrics are not dominated by a few majority classes; unrepresentative sampling otherwise biases the estimate [11].
- R5. **Sufficient per-class support.** At least three distinct CVEs are retained per CWE, so that macro-averaged CWE recall is computed over enough groups to be stable rather than driven by a single vulnerability.
- R6. **Determinism.** A fixed seed and a fixed query ratio make the partition reproducible across runs.

Concretely, the split groups the pairs first by CWE and then by CVE. For every CVE with at least `min_pairs_per_cve` entries it moves a fraction of the pairs to the query set while retaining at least one in the index; CVEs represented by a single pair remain in the index only. A final safety pass drops any query whose CVE lost all of its index support, enforcing R2.

Why the split must be at the CVE level and not only at the CWE level. A CWE is a coarse category: a single weakness class such as CWE-787 (out-of-bounds write) aggregates thousands of semantically unrelated defects, so same-CWE membership is far too weak to serve as retrieval ground truth. The precise positive is the *same underlying vulnerability*, i.e. the same CVE. This distinction has two practical consequences that a CWE-only split cannot address. First, a single CVE typically spans several files and methods that share one root cause and a near-identical fix. If pairs are assigned to the index and query sets by CWE alone — randomly, without regard to their CVE — sibling pairs of the same CVE straddle the boundary, and each query is then answered by retrieving its own near-duplicate. This is exactly the near-duplicate leakage shown to inflate vulnerability-detection results [12, 13], and it rewards memorisation over structural generalisation. Second, and symmetrically, an unconstrained CWE-level split can place *all* pairs of a query’s CVE outside the index, leaving that query with no valid positive and deflating (or undefining) its $\text{Hit}@k$ and MRR. Grouping by CVE — and only then partitioning *within* each CVE while keeping at least one pair in the index — is the unique unit of division that simultaneously prevents trivial sibling leakage and guarantees every query a legitimate, non-identical positive; neither property is enforceable at the CWE granularity.

Building on this leakage-safe partition, we additionally compare two CVE-aware samples of the corpus — one that preserves the natural CWE distribution and one that balances it across classes — to quantify how much the dataset’s class distribution, rather than the embedding itself, drives the observed retrieval scores.

2 Experiments

1. **Slicing:** *full* (the complete pre-patch CPG, G_{before}) vs. *sliced* (the fingerprint-diff subgraph G_{vuln} , retaining only nodes within 3 flow-edge hops of changed nodes).

2. **Diff labelling:** *absent* (all `diff` and `diff_weight` node attributes stripped) vs. *present* (each node annotated with its change class and a weight $w \in \{1.0, 0.8, 0.6, 0.2\}$ for `removed`, `fix_adjacent`, `edge_changed`, and `context` respectively).

This yields four graph representations:

Symbol	Sliced?	Labels?	Construction
G_{before}	No	No	Full pre-patch CPG; diff attributes removed.
$G_{\text{before}}^{+\ell}$	No	Yes	Full pre-patch CPG; diff labels transferred from G_{vuln} ; unmatched nodes default to <code>context</code> ($w=0.2$).
$G_{\text{vuln}}^{-\ell}$	Yes	No	Fingerprint-sliced subgraph; diff attributes stripped.
G_{vuln}	Yes	Yes	Fingerprint-sliced subgraph with native diff labels and weights.

2.0.1 Experimental Configurations

The ablation was run under three configurations that differ in *query protocol* and *PCA fitting procedure*, allowing us to disentangle representation quality from methodological artefacts.

	Run A	Run B	Run C
Script	<code>slicing_comparison</code>	<code>slicing_comparison</code>	<code>runner.py</code>
Query protocol	Symmetric: query graph uses the same variant as the index.	Asymmetric: augmented queries use G_{before} ; original queries use G_{vuln} .	Same as Run B.
PCA fitting	Index-only: <code>embed_many</code> on index, then on queries (PCA reused).	Same as Run A.	Index-only: PCA fitted via <code>embed_many</code> on index; queries projected via <code>embed_one</code> .
Graph variants	All four.	All four.	G_{vuln} only.

Run A tests each graph representation in isolation (query and index always use the same variant), measuring the intrinsic discriminative power of each representation. Runs B and C use the *asymmetric* protocol that reflects a realistic deployment scenario: augmented (LLM-rewritten) queries lack a patched version and can only provide the full pre-patch CPG, while original CVE queries have access to the vulnerability-sliced subgraph. Runs B and C differ only in the embedding call path (`embed_many` vs. `embed_one` for queries), allowing a direct comparison of batch vs. single-query embedding under identical data and PCA protocol.

Discussion

Same-CWE Retrieval (n=109 index, 27 query)

Embedder	Hit@1	Hit@5	MRR	CWE Recall
GIN	0.259	0.778	0.315	0.202
Combined	0.111	0.778	0.223	0.189
CodeBERT (baseline)	0.080	0.1518	0.098	0.141
CodeBERT (seq)	0.556	0.926	0.555	0.264
CodeBERT (pat)	0.519	0.889	0.549	0.276

Deliverables

- Source code of the implementation of the RAG-based multi-agent framework.

- The curated benchmark and/or CVEs, vulnerable code samples used for benchmarking, and any further data used to populate the knowledge base

References

- [1] International Organization for Standardization, “Iso/iec 27001:2022 information security, cybersecurity and privacy protection — information security management systems — requirements,” standard, International Organization for Standardization, Geneva, Switzerland, 2022.
- [2] A. Mahmud, Y. Rawajfih, and F. Wu, “A systematic evaluation of ai-generated security patches: Analyzing and exploiting ai-generated fixes,” in *2025 IEEE International Conference on Artificial Intelligence Testing (AITest)*, pp. 224–231, 2025.
- [3] K. McClanahan, S. Elder, M. L. Uwibambe, Y. Liu, R. Heng, and Q. Li, “When chatgpt meets vulnerability management: The good, the bad, and the ugly,” in *2024 International Conference on Computing, Networking and Communications (ICNC)*, pp. 664–670, 2024.
- [4] T. ZeMicheal, H. Chen, S. Davis, R. Allen, M. Demoret, and A. Song, “Llm agents for vulnerability identification and verification of cves,” in *CAMLIS’24: Conference on Applied Machine Learning for Information Security*, 2024.
- [5] C. Guo, Y. Nie, C. Xie, Z. Lin, W. Guo, and B. Li, “Bluecodeagent: A blue teaming agent enabled by automated red teaming for codegen ai,” 2025.
- [6] I. Bouzenia, P. Devanbu, and M. Pradel, “Repairagent: An autonomous, llm-based agent for program repair,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2188–2200, IEEE, 2025.
- [7] Joern Project, “Joern – the bug hunter’s workbench,” 2024.
- [8] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, “Netlsd: Hearing the shape of a graph,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD ’18, (New York, NY, USA), p. 2347–2356, Association for Computing Machinery, 2018.
- [9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [10] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 6, no. 4, pp. 1–21, 2012.
- [11] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *31st USENIX Security Symposium (USENIX Security 22)*, pp. 3971–3988, 2022.
- [12] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?,” *IEEE Transactions on Software Engineering*, 2022.
- [13] R. Croft, M. A. Babar, and M. M. Kholoosi, “Data quality for software vulnerability datasets,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pp. 121–133, 2023.